

LOVELY PROFESSIONAL UNIVERSITY

upGrad

School of Computer Science and Engineering

Lovely Professional University, Phagwara, Punjab (India)

SEM: 07

Year: 2026

AirAware — Air Pollution Early Warning

Deep Learning-Based Air Quality Index (AQI) Forecasting and Early Warning System Using LSTM and GRU Networks

Submitted By: Prabhu Pritam Maharana

Roll no: RK2P23UPA29

Reg no: 12302414

GitHub Link:

https://github.com/LEOPRITAM01/AirAware-Air_Pollution_Early_Warning

SUPERVISOR'S CERTIFICATE

This is to certify that the project report entitled "**AirAware — Air Pollution Early Warning System**", submitted by **Prabhu Pritam Maharana** (Roll No. RK2P23UPA29, Registration No. 12302414), B.Tech in Computer Science and Engineering (specialization in Data Science and Machine Learning), at **Lovely Professional University, Phagwara, India**, presents a record of his project work on deep learning-based AQI forecasting using LSTM and GRU networks, submitted for faculty evaluation. This work has not been submitted elsewhere to any other degree.

Karan Kumar Das

Date of Submission: 15 October 2026

Acknowledgement

I express my sincere gratitude to Lovely Professional University, Phagwara, and the School of Computer Science and Engineering for providing the academic framework within which this project report on **AirAware — Air Pollution Early Warning System** was prepared, as part of the B.Tech programme in Computer Science and Engineering with specialization in Data Science and Machine Learning.

I thank my faculty supervisor, Karan Kumar Das, for the opportunity to undertake and present this project in the Semester 07 academic programme.

I also acknowledge the open resources on which this work builds: the India Air Quality dataset published on Kaggle, and the open-source Python ecosystem, including pandas, NumPy, scikit-learn, TensorFlow/Keras and Streamlit, whose tools and documentation made the experiments and the dashboard possible.

Name of Candidate: Prabhu Pritam Maharana

Registration Number: 12302414

Table of Contents

1. Introduction **6**
- 1.1 Air Pollution and the Air Quality Index 6
- 1.2 Historical Analysis, Forecasting and Early Warning 6
- 1.3 About AirAware 6

2. Problem Statement **6**

3. Objectives **7**

4. Dataset Description **7**
- 4.1 Source and Role of city_day.csv 7
- 4.2 Structure of the File 7
- 4.3 Selected City and Series Used 8

5. Data Understanding and Exploratory Analysis **8**
- 5.1 Overview of the Dataset 8
- 5.2 Distribution of AQI 8
- 5.3 Differences Between Cities 9
- 5.4 Temporal Behaviour of the Delhi Series 9

6. Data Quality and Preprocessing **10**
- 6.1 Selection and Type Conversion 10
- 6.2 Missing AQI Values and Duplicate Dates 11
- 6.3 Daily Frequency and Interpolation 11
- 6.4 Data Limitations 11

7. Time-Series Preparation and Feature Engineering **11**
- 7.1 The 30-Reading Lookback Window 11
- 7.2 Scaling 12
- 7.3 Chronological Train/Test Split 12
- 7.4 Protection Against Information Leakage 13

8. Persistence Baseline **13**

9. LSTM Model **14**
- 9.1 Concept 14
- 9.2 Configuration Used in AirAware 14

10. GRU Model **15**
- 10.1 Concept 15
- 10.2 Configuration Used in AirAware 15

11. Training Methodology **15**
- 11.1 Training and Validation Procedure 15
- 11.2 Why Chronological Splitting Suits Time-Series Data 16
- 11.3 Saved Model Artifacts 16

12. Model Evaluation and Comparison **16**
- 12.1 Evaluation Metrics 16
- 12.2 Results 17
- 12.3 Interpretation 17
- 12.4 Why the Persistence Baseline Remains Important 18
- 12.5 Limitations of the Experiment and Possible Reasons 18
- 12.6 Note on Reproducibility of Reported Values 18

13. AirAware Streamlit Dashboard **18**
- 13.1 Overview 18
- 13.2 Layout and Controls 19
- 13.3 Model Performance Section 19
- 13.4 CSV Upload and Input Requirements 19
- 13.5 Data Validation and Quality Information 20
- 13.6 Historical AQI Trend Explorer 20
- 13.7 Recent Descriptive Statistics 20
- 13.8 Model Selection and Next-Day Forecast 20
- 13.9 Indicative AQI Category 21

13.10 Forecast Summary Download....21
13.11 Dashboard Limitations....21
14. System Architecture and Workflow....22
14.1 Offline Training Workflow....22
14.2 Dashboard Inference Workflow....23
14.3 Project Components....23
15. Results and Discussion....23
15.1 Summary of Findings....23
15.2 What the Results Do and Do Not Demonstrate....23
15.3 Dashboard Behaviour....23
16. Limitations and Responsible Use....23
17. Conclusion and Future Scope....24
17.1 Conclusion....24
17.2 Future Scope....24
18. Tools and Technologies....24
19. References....25

1. Introduction

1.1 Air Pollution and the Air Quality Index

Air pollution is the presence in the atmosphere of gases and fine particles at concentrations that can harm human health and the environment. Pollutants of common concern include fine particulate matter (PM2.5), coarse particulate matter (PM10), nitrogen dioxide (NO2), sulphur dioxide (SO2), carbon monoxide (CO), ozone (O3) and ammonia (NH3). Because several pollutants are measured in different units and have different health effects, public agencies summarise them into a single number called the Air Quality Index (AQI).

In India, the National Air Quality Index published by the Central Pollution Control Board (CPCB) groups AQI values into six categories: Good, Satisfactory, Moderate, Poor, Very Poor and Severe [1]. A higher AQI indicates poorer air quality. The index is useful because it turns a collection of concentration readings into a value that citizens, researchers and administrators can compare across days and across cities.

1.2 Historical Analysis, Forecasting and Early Warning

Daily AQI readings recorded over several years form a time series. Studying such a series shows how air quality behaves over time, including seasonal rises and falls, short spells of very high pollution and the day-to-day persistence of conditions. Time-series forecasting uses this history to estimate future values [2]. An early-warning system builds on a forecast: if tomorrow's AQI is expected to be high, residents and authorities could, in principle, be alerted in advance.

Recurrent neural networks such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks are designed for sequential data and are widely used for time-series problems [3, 4]. Whether they actually improve on very simple forecasting rules for a given dataset is an empirical question, and this report treats it as one.

1.3 About AirAware

AirAware — Air Pollution Early Warning System is an educational research prototype developed for this project. It uses historical daily AQI data for Delhi from the India Air Quality dataset on Kaggle [5] to train two recurrent models (LSTM and GRU) that predict the next day's AQI from the previous 30 daily readings. The models are compared with a persistence baseline, and a Streamlit dashboard [6] allows a user to upload recent AQI readings and obtain an experimental next-day forecast. The complete source code is available in the project repository [7].

AirAware is not an official air-quality monitoring service. Its outputs are experimental, are not medically validated, and must not replace official air-quality reports or health guidance.

2. Problem Statement

The problem addressed in this project is to anticipate the next day's Air Quality Index from a short history of past daily readings. Air quality in a large city such as Delhi varies strongly through the year (see Section 5), and daily values can change by tens or even hundreds of AQI points within a few days. A forecasting approach must therefore capture both the persistence of conditions from one day to the next and any longer-range structure in the recent history.

A second part of the problem concerns honest evaluation. A deep-learning model can look impressive without being useful if it is not compared with a simple reference. The most basic reference is the persistence baseline, which predicts that tomorrow's AQI will equal today's. For a series with strong day-to-day persistence, this baseline can be difficult to beat. The project therefore asks two questions: (i) how well can LSTM and GRU models forecast next-day AQI for Delhi from a 30-day window, and (ii) do they improve on the persistence baseline under a chronological held-out evaluation?

Finally, the project needs a way to present the forecasts responsibly. The results must be communicated through an interface that makes clear that the forecasts are experimental and do not replace official sources.

3. Objectives

The objectives of the project are:

- **Data preparation:** load the India Air Quality data, select the Delhi AQI series, and prepare it as a clean, regular daily time series.
- **Exploratory analysis:** examine the AQI distribution, differences between cities, and the temporal behaviour of the Delhi series.
- **Sequential modelling:** convert the series into supervised input sequences using a 30-day lookback window and a next-day target.
- **Model training:** train LSTM and GRU networks using a chronological training procedure that avoids information leakage.
- **Baseline comparison:** implement a persistence baseline and compare the learned models against it.
- **Evaluation:** assess all models with Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) on a held-out, chronologically later test period.
- **Dashboard development:** build an interactive Streamlit dashboard that validates uploaded AQI data, shows recent trends and statistics, and produces an experimental next-day forecast.
- **Responsible communication:** present forecasts with clear disclaimers, indicative category labels and an explicit comparison with the persistence baseline.

4. Dataset Description

4.1 Source and Role of city_day.csv

The data come from the *Air Quality Data in India* dataset published on Kaggle [5] (https://www.kaggle.com/datasets/rohanrao/air-quality-data-in-india). The dataset contains several files; the initial AirAware experiment uses only `city_day.csv`, which holds daily city-level pollutant measurements and the corresponding AQI. All values are historical observations. The dashboard does not fetch live measurements from any sensor or website.

4.2 Structure of the File

The notebook confirms that `city_day.csv` contains 29,531 rows and 16 columns, covering 26 cities and the period from 1 January 2015 to 1 July 2020. The columns are listed in Table 1 together with the number of

missing values reported by the notebook over the whole file. AQI is available for 24,850 rows; the remaining 4,681 rows have no AQI (and no AQI_Bucket).

Table 1: Columns of city_day.csv and missing-value counts (all cities, as reported by the notebook).

Column	Content	Missing values
City	City name (26 distinct cities)	0
Date	Observation date (daily)	0
PM2.5, PM10	Particulate matter concentrations	4,598; 11,140
NO, NO2, NOx, NH3	Nitrogen oxides and ammonia	3,582; 3,585; 4,185; 10,328
CO, SO2, O3	Carbon monoxide, sulphur dioxide, ozone	2,059; 3,854; 4,022
Benzene, Toluene, Xylene	Volatile organic compounds	5,623; 8,041; 18,109
AQI	Air Quality Index value (numeric)	4,681
AQI_Bucket	AQI category label	4,681

4.3 Selected City and Series Used

The experiment uses the Delhi subset only. Delhi has 2,009 daily rows in the file, of which 1,999 contain an AQI value. The notebook's per-city summary shows that Delhi has the largest number of AQI records among the 26 cities, which is consistent with its choice for a model that needs a long, continuous series. Only the `Date` and `AQI` columns are used for modelling; the individual pollutant columns are not model inputs. The model is therefore a univariate forecaster of AQI from past AQI.

The model, scaler and metadata saved by the notebook belong to Delhi. Consequently, the dashboard's forecasts are meaningful only as an experiment with Delhi-like data; they have not been validated for other cities.

5. Data Understanding and Exploratory Analysis

5.1 Overview of the Dataset

The notebook first inspects the shape, data types, missing values, date range and city list of `city_day.csv`. The `Date` column is initially read as text and is converted to a date type. Missingness is substantial for several pollutant columns, with Xylene (18,109 missing values) and PM10 (11,140) the most affected, while AQI itself is missing for 4,681 of the 29,531 rows (Table 1). Because AQI is the target variable, only the availability of AQI directly affects this project.

5.2 Distribution of AQI

Figure 1 shows the distribution of all available AQI values across all cities. Most observations lie in the lower part of the scale, with a pronounced right tail. Computing summary statistics from the supplied file confirms this shape: the mean AQI over the 24,850 available values is about 166, the median is 118, and the maximum is 2,049. A mean well above the median is characteristic of a right-skewed distribution. Values above 500 appear in the data even though the Indian AQI categories end at 500; the reasons were not investigated in this project.

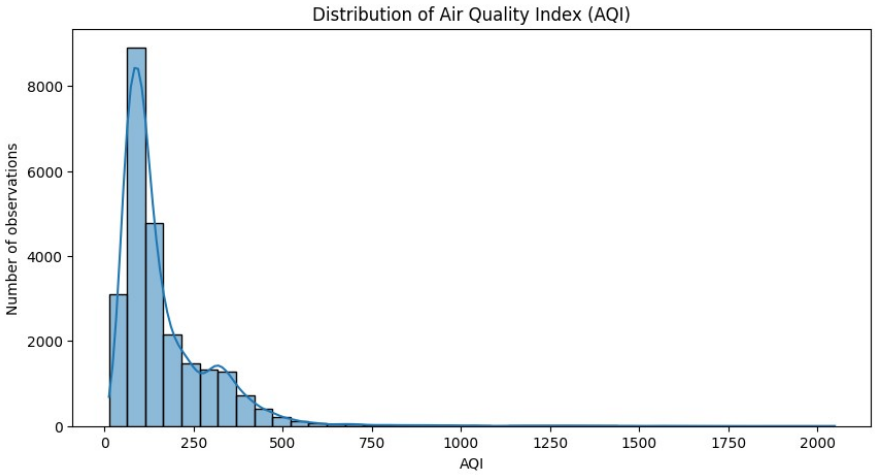


Figure 1: Distribution of AQI values across all cities in city_day.csv (notebook output).

5.3 Differences Between Cities

Figure 2 compares the average AQI by city. Ahmedabad has the highest average (about 452), followed by Delhi (about 259), Patna (about 241), Gurugram (about 225) and Lucknow (about 218); Aizawl has the lowest average (about 35). These averages should be read with care, because the number of available AQI days differs greatly between cities: Delhi has 1,999 and Ahmedabad 1,334, whereas Aizawl has only 111, so cities are not observed over comparable periods.

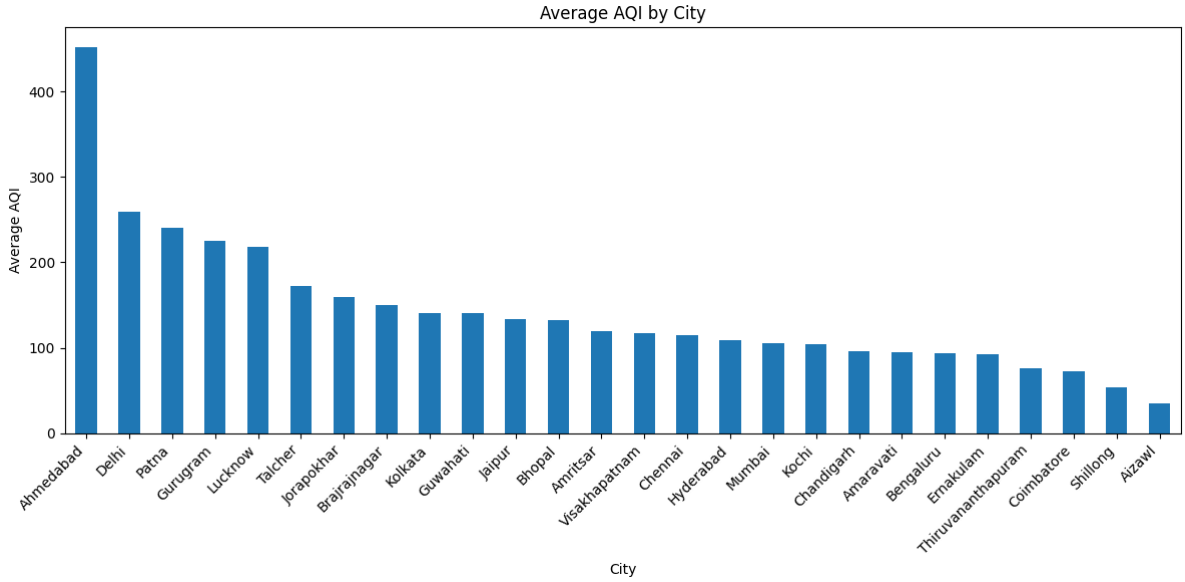


Figure 2: Average AQI by city (notebook output). The number of available days differs between cities.

5.4 Temporal Behaviour of the Delhi Series

Figure 3 plots the Delhi AQI series from January 2015 to July 2020. The series is noisy from day to day and shows recurring periods of high values in each year, with occasional sharp peaks. After preparation (Section 6) the series has 2,009 daily values with a minimum of 29, a maximum of 716 and a mean of about 259.

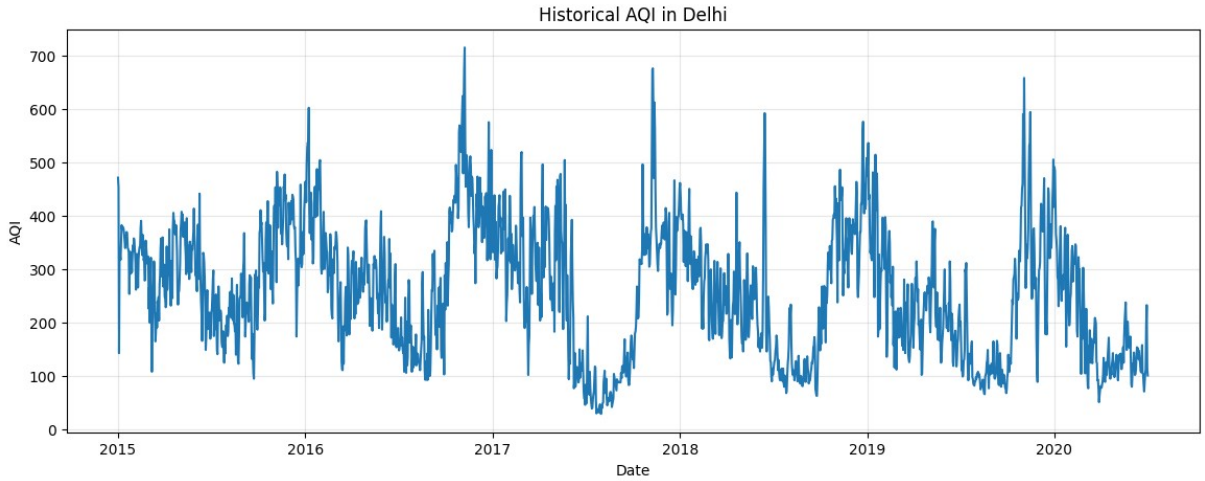


Figure 3: Historical daily AQI for Delhi (notebook output).

To quantify the recurring pattern visible in Figure 3, the monthly mean of the prepared Delhi series was computed from the supplied file for this report and is shown in Figure 4. Mean AQI is highest in November (about 402), December (about 371) and January (about 360), and lowest in August (about 131) and July (about 138). The series therefore shows a clear seasonal pattern within the observed period. This chart was generated for this report from the same prepared series and is not part of the original notebook; no explanation for the seasonality (for example meteorology or emission sources) was analysed in the project.

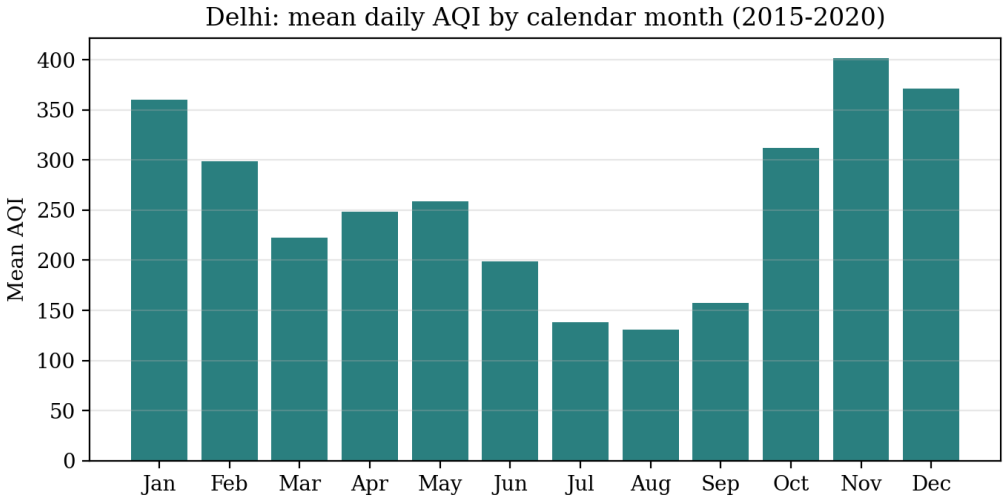


Figure 4: Mean daily AQI by calendar month for the prepared Delhi series (2015–2020), generated from city_day.csv.

6. Data Quality and Preprocessing

The preprocessing steps below are those performed in the notebook; each is summarised in Table 2.

6.1 Selection and Type Conversion

The rows for Delhi are selected and reduced to the two columns `Date` and `AQI`. `Date` is parsed with `pd.to_datetime(..., errors="coerce")` and `AQI` is converted with `pd.to_numeric(..., errors="coerce")`, so that any malformed entries become missing values instead of causing errors.

6.2 Missing AQI Values and Duplicate Dates

Rows with a missing date or missing AQI are dropped. The remaining rows are grouped by date and averaged, which guarantees one value per date even if duplicate dates exist, and are then sorted chronologically. After this step the Delhi series contains 1,999 records spanning 1 January 2015 to 1 July 2020. When the supplied file was checked during preparation of this report, no duplicate Delhi dates were found, so the averaging step did not change any value.

6.3 Daily Frequency and Interpolation

Recurrent models assume that consecutive inputs are consecutive days. The series is therefore re-indexed to a strict daily frequency, which exposes 10 calendar days that had no AQI value. These interior gaps are filled by time-based linear interpolation (`interpolate(method="time", limit_area="inside")`). Interpolation is restricted to gaps inside the series, so no values are extrapolated at either end. After a final removal of any remaining missing values (none remained), the series contains 2,009 daily records with no missing AQI.

```
city_df = city_df.set_index("Date").asfreq("D")
city_df["AQI"] = city_df["AQI"].interpolate(method="time", limit_area="inside")
city_df = city_df.dropna().reset_index()
```

Table 2: Summary of the preprocessing workflow for the Delhi series.

Step	Operation	Result
1	Select Delhi; keep Date and AQI	2,009 rows (1,999 with AQI)
2	Parse dates; coerce AQI to numeric	Malformed values become missing
3	Drop missing; average duplicate dates; sort by date	1,999 records
4	Re-index to daily frequency	10 missing days exposed
5	Time-based interpolation (interior gaps only)	2,009 daily records, 0 missing

6.4 Data Limitations

Interpolation was applied to the whole series before the chronological split. Only 10 values are affected, so the practical effect is negligible, but a stricter pipeline would interpolate within the training and test segments separately. The interpolated values are estimates rather than measurements. The notebook did not apply any outlier removal: extreme AQI values are retained and enter both training and evaluation. The pollutant columns, which contain many more missing values than AQI, were not used.

7. Time-Series Preparation and Feature Engineering

7.1 The 30-Reading Lookback Window

Each training example consists of 30 consecutive daily AQI values (the input window) and the AQI of the following day (the target). The lookback of 30 days is stored in the metadata file as `lookback = 30`. Figure 5 illustrates one such sequence taken from the Delhi series: the 30 highlighted points form the input and the star marks the value to be predicted. The window slides forward one day at a time, so a series of N daily values yields N − 30 examples.

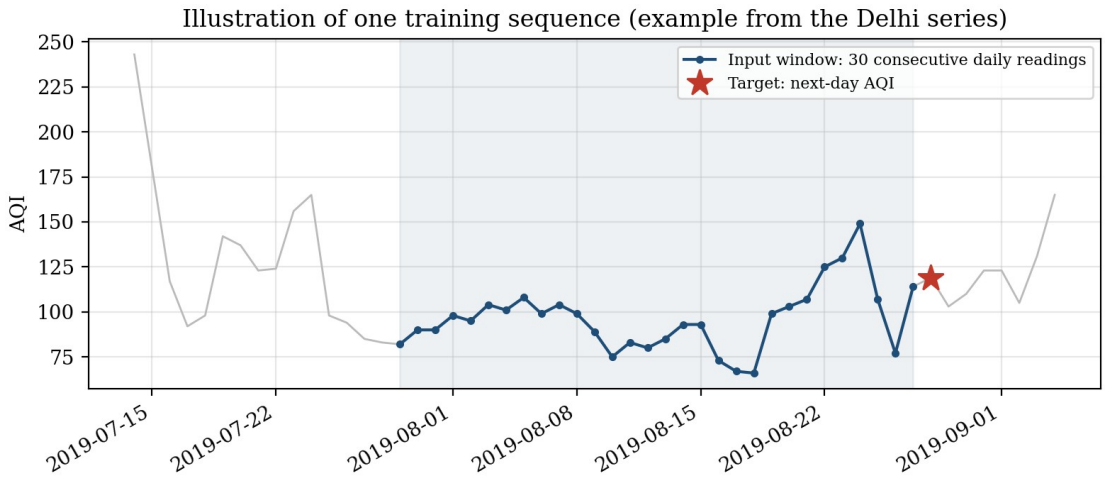


Figure 5: Illustration of one input sequence (30 days) and its next-day target, drawn from the prepared Delhi series.

```
LOOKBACK = 30
def create_sequences(data, lookback=30):
    X, y = [], []
    for i in range(lookback, len(data)):
        X.append(data[i-lookback:i, 0])
        y.append(data[i, 0])
    return np.array(X), np.array(y)
```

7.2 Scaling

AQI values are scaled to the range 0–1 with scikit-learn's `MinMaxScaler` [8]. The scaler is **fitted only on the training portion** of the series (the first 1,607 daily values) and then applied to the whole series. Predictions are converted back to AQI units with the inverse transform before any error is computed, so all reported errors are in AQI points. The scaler is saved as `scaler.pkl` so the dashboard applies the same transformation to uploaded data.

7.3 Chronological Train/Test Split

The series is split chronologically: the first 80% of the daily values (index 1,607 onward is the boundary) is used for training and the final 20% for testing. Each example is assigned to the training or test set according to the date of its *target*. Figure 6 shows the two periods and Table 3 gives the resulting sizes. The test targets cover 27 May 2019 to 1 July 2020 (402 days). Input windows for the earliest test examples reach back into the last days of the training period, which is normal for a sliding window and does not use any information from the future.

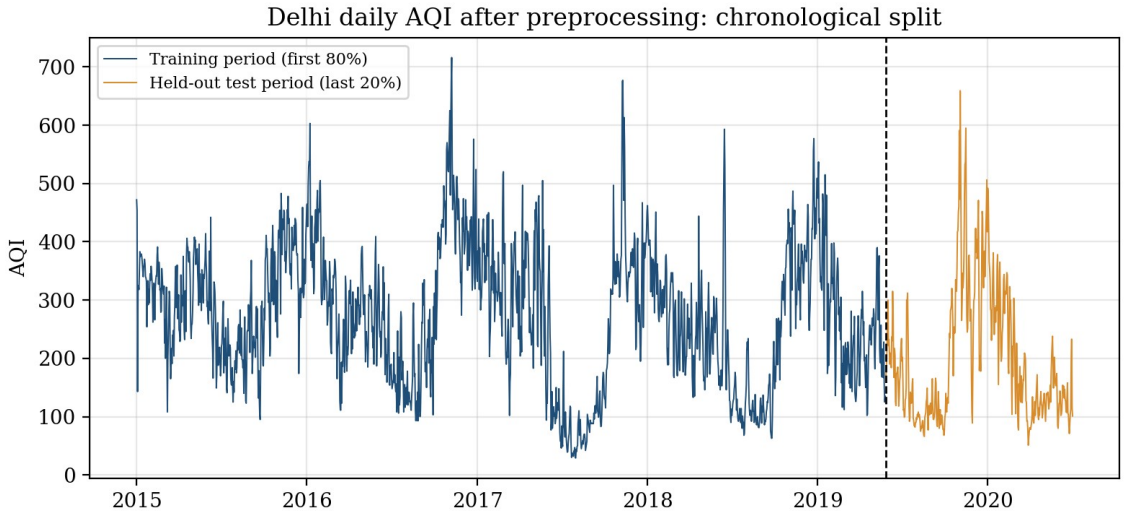


Figure 6: Chronological split of the prepared Delhi series into training and held-out test periods.

Table 3: Sequence counts and shapes produced by the notebook.

Item	Value
Daily records after preparation	2,009
Split index (80%)	1,607
Training sequences	1,577 (input shape 1,577 × 30 × 1)
Test sequences	402 (test targets 27 May 2019 – 1 July 2020)
Lookback window	30 daily readings
Features per time step	1 (AQI only)

7.4 Protection Against Information Leakage

Several design choices prevent the models from seeing test information during training: (i) the split is chronological, not random, so no future day is used to predict a past day; (ii) the scaler is fitted on training data only, so test-period minima and maxima do not influence scaling; (iii) the validation subset used during training is also taken from the end of the training sequences without shuffling; and (iv) the test sequences are used only for final evaluation. The one exception noted in Section 6.4 is the interpolation of 10 interior gaps before the split, which is a minor and clearly stated limitation.

8. Persistence Baseline

The persistence baseline predicts that the AQI on the next day will equal the most recent observed AQI. For each test window, the prediction is simply the last of its 30 input values:

```
baseline_scaled = X_test[:, -1, 0].reshape(-1, 1)
baseline_predictions = scaler.inverse_transform(baseline_scaled).ravel()
```

Despite its simplicity, this baseline is essential. It requires no training and no parameters, and it encodes the strong day-to-day persistence typical of air-quality series. If a trained neural network cannot beat it, the extra complexity is not justified by the evidence. On the held-out test period the baseline obtains an MAE of 31.46 and an RMSE of 44.97. These two values were also recomputed independently from the supplied `city_day.csv` during preparation of this report and agree with the saved results.

Figure 7 compares the actual test-period AQI with the baseline prediction. The baseline curve follows the actual curve closely but lags it by one day, which is exactly what a persistence rule produces: it reacts to a rise or fall only after it has happened. The baseline is therefore accurate when conditions change slowly and weak when AQI changes sharply.

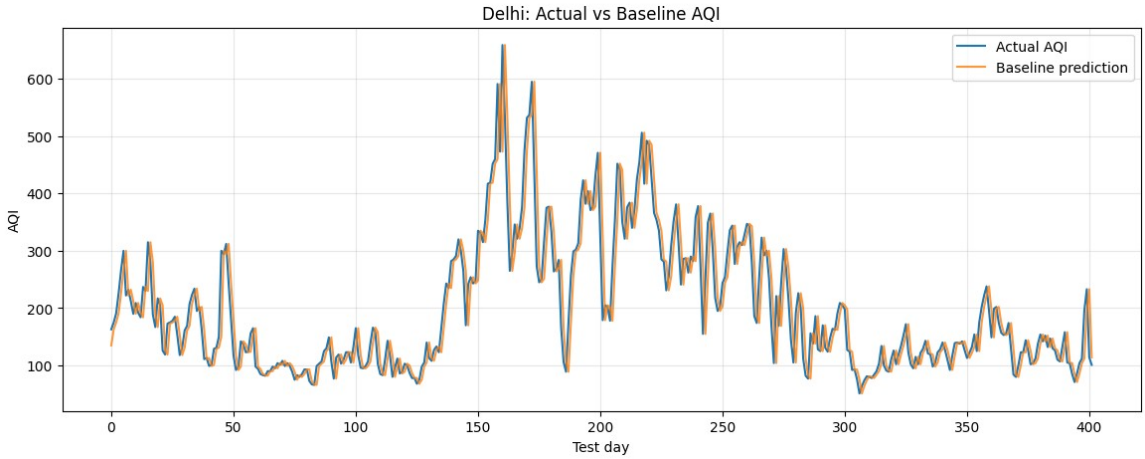


Figure 7: Actual AQI versus persistence-baseline prediction over the 402-day test period (notebook output).

9. LSTM Model

9.1 Concept

The Long Short-Term Memory network was introduced to address the difficulty plain recurrent networks have in learning dependencies across many time steps [3]. An LSTM unit maintains a *cell state*, a running memory that is passed forward through the sequence, together with a hidden state. Three gates, each a learned function of the current input and previous hidden state, regulate this memory: the **forget gate** decides which part of the stored memory to discard, the **input gate** decides which new information to write, and the **output gate** decides how much of the memory to expose as the unit's output. In forecasting, this allows the network to keep information from earlier in the 30-day window that remains relevant for tomorrow's value while ignoring information that does not.

9.2 Configuration Used in AirAware

The LSTM model in the notebook is a Keras `Sequential` network [9] with two stacked LSTM layers, dropout regularisation [10] and a small dense output head, as listed in Table 4. The input shape is (30, 1): 30 time steps with a single feature.

Table 4: LSTM architecture as defined in the notebook.

Layer	Setting
LSTM	64 units, return_sequences = True, input shape (30, 1)
Dropout	rate 0.2
LSTM	32 units
Dropout	rate 0.2
Dense	16 units, ReLU activation
Dense (output)	1 unit (scaled next-day AQI)

10. GRU Model

10.1 Concept

The Gated Recurrent Unit is a more compact gated recurrent architecture introduced for sequence modelling [4] and evaluated against LSTM on sequence tasks in later work [11]. A GRU has no separate cell state; it keeps a single hidden state controlled by two gates. The **update gate** decides how much of the previous hidden state is carried forward and how much is replaced by new information. The **reset gate** decides how much of the previous state is used when computing the new candidate state. Having fewer gates and parameters than an LSTM, a GRU is often quicker to train and can perform comparably, although neither architecture is universally better.

10.2 Configuration Used in AirAware

The GRU model uses exactly the same layer structure and hyperparameters as the LSTM, with the two recurrent layers replaced by GRU layers (Table 5). Keeping everything else identical means that differences between the two models are attributable to the recurrent cell type rather than to a different design, although both models were trained only once and no repeated-run comparison was made.

Table 5: GRU architecture as defined in the notebook.

Layer	Setting
GRU	64 units, return_sequences = True, input shape (30, 1)
Dropout	rate 0.2
GRU	32 units
Dropout	rate 0.2
Dense	16 units, ReLU activation
Dense (output)	1 unit (scaled next-day AQI)

11. Training Methodology

11.1 Training and Validation Procedure

Both models are trained on the 1,577 training sequences with the settings in Table 6. During training, Keras reserves the last 10% of the training sequences as a validation set (`validation_split = 0.1`) and does not shuffle the data (`shuffle = False`). Because the reserved portion is the chronologically latest part of the training sequences, validation loss measures performance on later data than the data used for fitting, which is more realistic for a forecasting task. An `EarlyStopping` callback monitors the validation loss and stops training when it has not improved for 5 consecutive epochs, restoring the weights from the best epoch. Random seeds for TensorFlow and NumPy are set to 42 before each model is built.

Table 6: Training configuration (identical for LSTM and GRU).

Setting	Value
Optimiser	Adam [12] (Keras default settings)
Loss function	Mean squared error (MSE) on scaled AQI
Maximum epochs	30
Batch size	32
Validation split	0.1 (last 10% of training sequences)
Shuffling	Disabled (shuffle = False)
Callback	EarlyStopping: monitor val_loss, patience 5, restore_best_weights = True
Random seed	42 (TensorFlow and NumPy)
Framework	TensorFlow / Keras [13, 14], TensorFlow 2.21.0 in the notebook

The number of epochs actually run before early stopping is not reported here, because the supplied notebook records a separate experiment from the one shown in the current dashboard (see Section 12.6).

11.2 Why Chronological Splitting Suits Time-Series Data

A random train/test split would place days from the same weeks and seasons in both sets and would let the model effectively interpolate between neighbouring days it has already seen. This overstates the model's ability to forecast the future. Chronological splitting mimics real use: the model is trained on the past and tested on a later period it has never seen. The cost is that the test period may differ in character from the training period, which is a limitation discussed in Section 12.5.

11.3 Saved Model Artifacts

After evaluation the notebook saves the artifacts in Table 7 into a folder named `airaware_model`, which the dashboard loads at start-up. The metadata file records the city, the lookback, the name of the model with the lowest MAE (`best_model`) and the six evaluation metrics.

Table 7: Files saved by the notebook and used by the dashboard.

File	Content
lstm_model.keras	Trained LSTM model
gru_model.keras	Trained GRU model
scaler.pkl	MinMaxScaler fitted on the training portion (saved with joblib)
metadata.json	City (Delhi), lookback (30), best_model, baseline/LSTM/GRU MAE and RMSE

12. Model Evaluation and Comparison

12.1 Evaluation Metrics

Predictions for the 402 test days are converted back to AQI units and compared with the observed values using two metrics. **Mean Absolute Error (MAE)** is the average absolute difference between predicted and observed AQI; it is easy to interpret as "the typical size of an error in AQI points". **Root Mean Squared Error (RMSE)** squares the errors before averaging and then takes the square root, so it penalises large errors

more strongly than MAE; a model that is usually accurate but occasionally badly wrong is penalised more under RMSE. For both metrics, lower is better.

12.2 Results

The results in this section are those displayed by the current AirAware dashboard (see Figure 9). They are given in Table 8 and illustrated in Figure 8.

Table 8: Test-set error of each model on the held-out Delhi test period (402 days), as displayed by the current dashboard. Lower is better.

Model	MAE	RMSE	MAE vs baseline	RMSE vs baseline
Persistence Baseline	31.46	44.97	–	–
LSTM	33.27	45.53	+1.81	+0.56
GRU	31.98	44.15	+0.52	−0.82

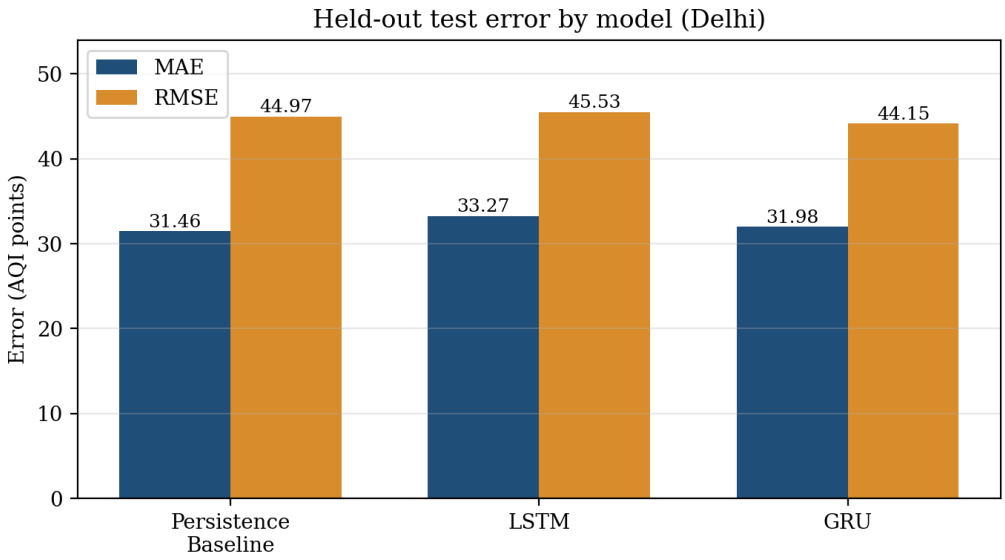


Figure 8: MAE and RMSE of the persistence baseline, LSTM and GRU on the held-out test period (current dashboard values).

12.3 Interpretation

- **Lowest MAE:** the persistence baseline (31.46). It has a smaller average absolute error than either learned model. The GRU is 0.52 points higher and the LSTM 1.81 points higher.
- **Lowest RMSE:** the GRU (44.15), 0.82 points below the baseline (44.97). The LSTM (45.53) is above the baseline on this metric as well.
- **LSTM versus baseline:** the LSTM does not outperform the persistence baseline on either metric.
- **Overall:** deep learning did **not** outperform the baseline overall. The only advantage of a learned model is the GRU's slightly lower RMSE, which suggests it makes somewhat smaller large errors, whereas the baseline is better on the typical error.
- **Size of differences:** all three models have errors of the same order (about 31–34 for MAE and 44–46 for RMSE). No statistical test was performed, so the small differences should not be interpreted as a significant ranking.

12.4 Why the Persistence Baseline Remains Important

Day-to-day AQI changes are strongly correlated, so "tomorrow equals today" is already a competitive forecast. A model is useful only to the extent that it improves on this reference. In AirAware the baseline is the benchmark that keeps the evaluation honest: the results show that, with the available inputs and settings, the recurrent models have not yet demonstrated a consistent gain over it. The dashboard therefore reports the baseline alongside the model forecast.

12.5 Limitations of the Experiment and Possible Reasons

The experiment has several limitations that bear on how the results should be read. These are possible factors only; none was tested in the project.

- **Single split and single run:** one chronological split and one training run per model were evaluated, so run-to-run variation in neural-network training is not quantified.
- **Limited inputs:** the models see only the past AQI. Weather, pollutant concentrations and calendar effects were not used.
- **No systematic tuning:** the architecture and hyperparameters were set once and not searched.
- **Modest data size:** the training set has 1,577 sequences, which is small for recurrent networks.
- **Test-period character:** the 402-day test period runs from May 2019 to July 2020. It may differ in pattern from the training period, and it includes early 2020, when unusual activity levels related to COVID-19 restrictions occurred in India. This was not analysed.
- **One-step horizon and sharp changes:** the target is one day ahead, where persistence is strongest, and a sudden jump or fall is hard for any model to anticipate from past AQI alone.

The results describe performance on a historical test period for Delhi. They do not guarantee similar performance on future data or in other cities.

12.6 Note on Reproducibility of Reported Values

The current dashboard displays the values shown in Table 8. The supplied notebook and metadata file contain a separate recorded experiment whose LSTM and GRU values differ from those in Table 8; the cause of the difference could not be conclusively established from the available evidence. The notebook values are not reported in this report and are not claimed to match the dashboard. All results and conclusions here are based on the values in Table 8.

13. AirAware Streamlit Dashboard

13.1 Overview

The dashboard is implemented in a single file, `app.py`, using Streamlit [6]. It loads the saved LSTM and GRU models, the scaler and the metadata from the `airaware_model` folder, and presents the model performance, a CSV upload for recent AQI readings, a data-quality panel, a trend explorer, recent statistics and the next-day forecast. Figure 9 shows the dashboard's landing view. In the project demonstration, the dashboard was served from Google Colab through a temporary public tunnel; this is a demonstration set-up and not a production deployment.

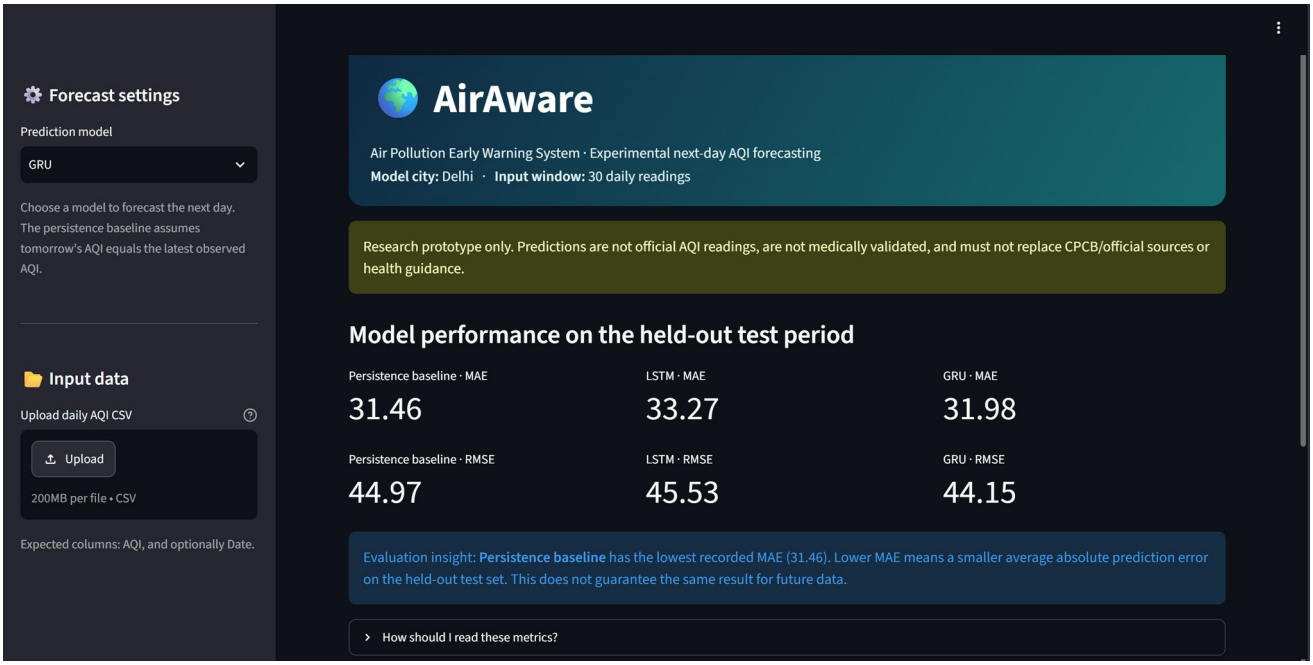


Figure 9: AirAware dashboard landing view: forecast settings and upload control (sidebar), project banner, research-prototype warning, and held-out test performance. Browser interface cropped out.

Only this landing view was available as a screenshot. The remaining sections below are described from the dashboard source code; they appear after a CSV file is uploaded and were not captured as screenshots.

13.2 Layout and Controls

The page uses a wide layout with an expanded sidebar. A banner at the top shows the project name, the tagline "Air Pollution Early Warning System · Experimental next-day AQI forecasting", the model city (Delhi) and the input window (30 daily readings), both read from the metadata file. A warning box states that the application is a research prototype, that its predictions are not official AQI readings, are not medically validated and must not replace CPCB/official sources or health guidance. The sidebar contains the **Prediction model** selector (GRU or LSTM; GRU is the default), a note explaining that the persistence baseline assumes tomorrow's AQI equals the latest observed AQI, and the CSV uploader.

13.3 Model Performance Section

The "Model performance on the held-out test period" section displays the MAE and RMSE of the persistence baseline, LSTM and GRU from the metadata file. An "Evaluation insight" box names the model with the lowest MAE and reminds the user that this does not guarantee the same result for future data. An expandable section, "How should I read these metrics?", explains MAE, RMSE and the persistence baseline and states that the metrics describe the historical test period, not the accuracy of a particular uploaded file.

13.4 CSV Upload and Input Requirements

Until a file is uploaded, the dashboard shows an instruction and an expandable example of the expected format (a `Date,AQI` header followed by rows) and stops. The uploaded file must contain an `AQI` column named exactly so; a `Date` column is optional. The checks performed on the upload are summarised in Table 9.

Table 9: Input validation performed by the dashboard.

Check	Behaviour
File cannot be read as CSV	Error message; processing stops
No column named AQI	Error message listing the columns found; processing stops
Non-numeric or missing AQI	Rows removed; the number removed is reported
Date column present	Dates parsed; rows with invalid dates reported; data sorted by date
No Date column	Row order is assumed to be chronological
Negative AQI values	Warning to verify the source data
AQI values above 1,000	Warning to verify units and source
Fewer than 30 valid readings	Error message; processing stops

13.5 Data Validation and Quality Information

After validation, four summary metrics are shown: the number of valid readings (with the number of invalid rows removed), the latest AQI in the file with its change from the previous row, the average of the 30-reading model window, and the range (minimum to maximum) of that window. An expandable "Data quality and preparation" section lists the rows read, the rows removed, the valid readings remaining, the dated range if dates exist, and states that the model uses the most recent 30 valid AQI values and that missing AQI rows are dropped without inferring missing measurements.

13.6 Historical AQI Trend Explorer

The "Trend explorer" tab draws two line charts: the full uploaded history, and the most recent 30-reading model window. If a usable `Date` column is present the horizontal axis is the date; otherwise it is the reading order, and the dashboard says so. A short text compares the first and latest value in the window: a rise or fall of more than 10 AQI points is reported as such, and a smaller net change is described as relatively small. The dashboard labels this as a descriptive comparison, not a formal trend test.

13.7 Recent Descriptive Statistics

The "Recent statistics" tab shows a table for the model window containing the count, mean, median, minimum, maximum, sample standard deviation, first value and latest value. A caption notes that the statistics refer to the most recent model input window, not necessarily the whole file.

13.8 Model Selection and Next-Day Forecast

The "Next-day forecast" tab applies the saved scaler to the latest 30 valid readings, passes them to the selected model (LSTM or GRU), and converts the output back to AQI units with the inverse transform. The predicted value is shown with its change from the latest reading. Because AQI cannot be negative, a negative raw output is floored at 0 and a note displays the raw value. The tab also shows the persistence forecast (the latest uploaded AQI) as a benchmark and a bar chart of the latest observed AQI, the persistence forecast and the model forecast. If the model forecast differs from the latest reading by more than 25 AQI points, an increase triggers a warning and a decrease a confirmation message, each described as an uncertain model signal and not an official warning or guarantee; smaller differences produce a neutral message. A one-paragraph summary sentence restates the forecast, the latest value and the baseline.

13.9 Indicative AQI Category

The predicted value is mapped to one of the six Indian AQI categories in Table 10, with a short descriptive sentence from the application. The bands follow the CPCB National AQI categories [1]. The label is described in the code and in the interface as indicative and experimental, because it is derived from a model forecast and not from a measurement.

Table 10: Indicative category bands used by the dashboard.

AQI range	Category	Dashboard description (abridged)
up to 50	Good	Air quality is generally satisfactory
51–100	Satisfactory	Sensitive people may wish to monitor symptoms
101–200	Moderate	Some people may experience discomfort, especially sensitive groups
201–300	Poor	Prolonged exposure may affect breathing for some people
301–400	Very Poor	Respiratory effects may occur with prolonged exposure
above 400	Severe	Potential health effects may occur even in otherwise healthy people

13.10 Forecast Summary Download

A button downloads the forecast as a one-row CSV file named `airaware_forecast_summary.csv`. Its columns are listed in Table 11; the final column carries a disclaimer that the forecast is experimental and not an official measurement or health advice.

Table 11: Columns of the downloadable forecast summary.

Column	Meaning
City	Model city from the metadata (Delhi)
Selected_Model	LSTM or GRU
Predicted_Next_Day_AQI	Forecast, rounded to two decimals
Experimental_Category	Indicative category
Last_Observed_AQI	Latest AQI in the uploaded data
Change_vs_Last_Reading	Forecast minus latest reading
Persistence_Baseline_Prediction	Persistence forecast (latest AQI)
Disclaimer	Experimental-forecast disclaimer text

13.11 Dashboard Limitations

A closing "About this prototype" section states the purpose, models and evaluation, and that the application forecasts from user-uploaded historical values and does not fetch live data. Reading the code also shows several limitations. The dashboard does not check that the uploaded readings are consecutive days, so gaps or irregular spacing are not detected. Rows with an invalid date are placed last when sorting and could therefore enter the model window. The model was trained on Delhi data, but no check confirms that an uploaded file refers to Delhi. Values outside the range seen in training are scaled beyond 0–1 and may be extrapolated unreliably. The forecast is a single point estimate without an uncertainty interval.

14. System Architecture and Workflow

AirAware consists of two connected workflows, shown in Figure 10. The **offline training workflow** (left, performed in the notebook) turns the historical dataset into saved model artifacts. The **dashboard inference workflow** (right, performed by `app.py`) loads those artifacts and applies them to data uploaded by the user.

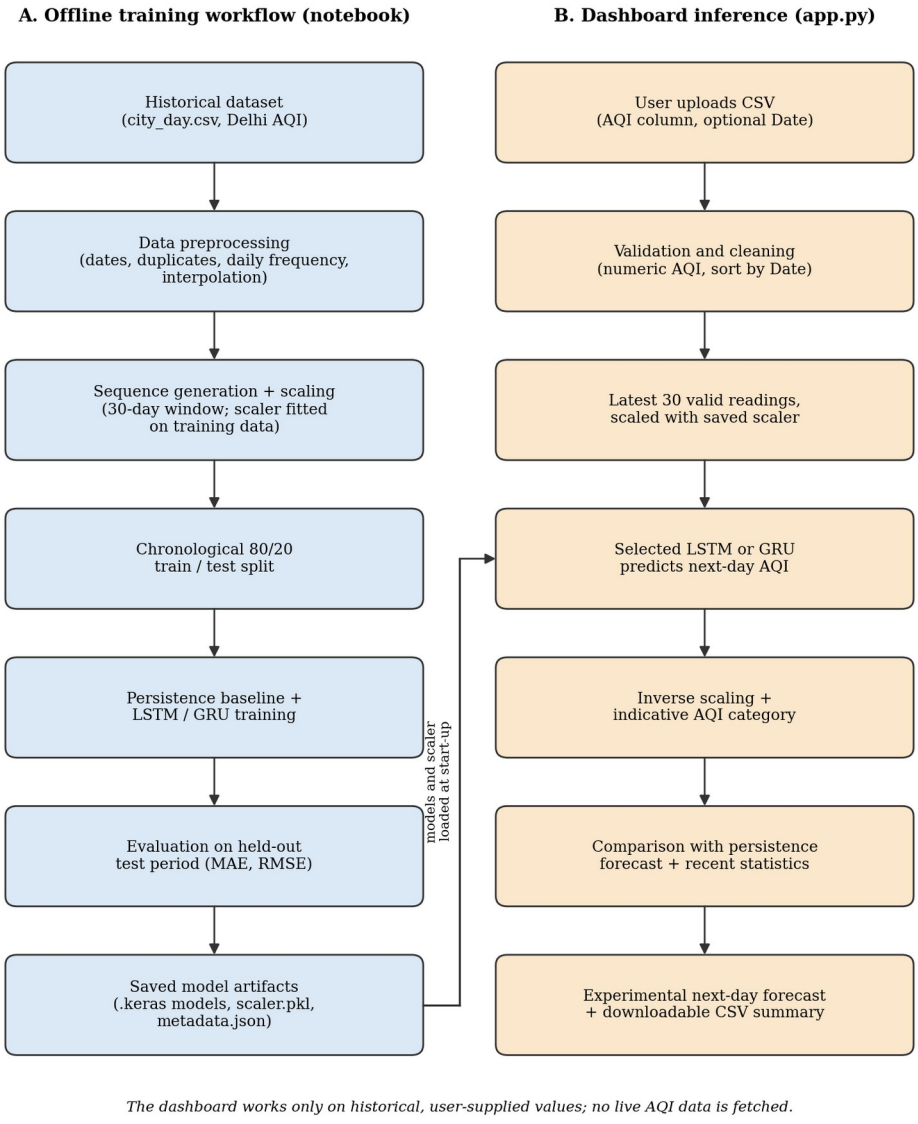


Figure 10: AirAware system architecture: offline training workflow (left) and dashboard inference workflow (right).

14.1 Offline Training Workflow

The historical dataset (`city_day.csv`) is preprocessed into a regular daily Delhi AQI series (Section 6). Sequences of 30 readings and next-day targets are generated, and the data are scaled with a scaler fitted on the training portion (Section 7). After a chronological 80/20 split, the persistence baseline is computed and the LSTM and GRU models are trained (Sections 8–11). All three are evaluated with MAE and RMSE on the held-out test period (Section 12), and the models, scaler and metadata are saved.

14.2 Dashboard Inference Workflow

When the dashboard starts it loads the saved models, scaler and metadata. A user uploads a CSV of recent daily AQI values; the file is validated and cleaned, and the latest 30 valid readings are scaled with the saved scaler. The selected model predicts the next day, the output is converted back to AQI units, an indicative category is assigned, and the forecast is shown next to the persistence forecast and recent statistics. The user can download a summary CSV. No training takes place in the dashboard, and no live data are retrieved.

14.3 Project Components

The project repository [7] contains the dashboard source (`app.py`), the saved models and metadata (`airaware_model`), the dependency list (`requirements.txt`) and a README describing how to run the application locally (`pip install -r requirements.txt` then `streamlit run app.py`). The experiment itself is in the Jupyter notebook `AirAware.ipynb`, which was run in Google Colab.

15. Results and Discussion

15.1 Summary of Findings

On the held-out Delhi test period, the persistence baseline achieved the lowest MAE (31.46), the GRU achieved the lowest RMSE (44.15), and the LSTM was behind the baseline on both metrics (Table 8). The practical conclusion is that, for next-day AQI in Delhi from a 30-day AQI-only window, the learned models were roughly on par with the persistence baseline but did not demonstrate a clear overall improvement over it.

15.2 What the Results Do and Do Not Demonstrate

The results demonstrate that a complete, leakage-aware forecasting pipeline was built and evaluated against an appropriate baseline, and that the GRU reduces the squared-error-based metric slightly relative to the baseline. They do not demonstrate that LSTM or GRU forecasting is better than persistence for this task, that the models would perform similarly on later data or in other cities, or that the forecasts are suitable for health decisions. The differences between models are small, were measured on a single run and split, and were not tested for statistical significance.

15.3 Dashboard Behaviour

The dashboard makes the comparison visible to the user: the baseline and the model forecast are always displayed together, the test-set metrics are shown with an explanation, and every forecast is accompanied by prototype and non-medical disclaimers. This design reflects the finding of the experiment: because the baseline is hard to beat, users should treat a model forecast as one experimental signal rather than as a reliable prediction.

16. Limitations and Responsible Use

- **Historical data only:** the models were trained on data from 2015 to mid-2020, and conditions, monitoring practices and emission patterns may have changed since.
- **Missing measurements:** about 4,700 AQI values are missing across the dataset, and 10 Delhi days were filled by interpolation; interpolated values are estimates.

- **Limited generalisation:** the models were trained and tested on one city and one 30-day, AQI-only input window. Their behaviour in other cities or with different inputs is unknown.
- **Forecasting error:** typical test errors are over 30 AQI points, which is large enough to change the category of a day. Forecasts of sudden pollution episodes are especially uncertain.
- **No live data integration:** the dashboard relies on user-uploaded files; it does not fetch current measurements and cannot detect stale or incorrect uploads.
- **No medical or regulatory validation:** the category labels and descriptions are indicative only, and no health-effect or regulatory validation was performed.
- **Not an official service:** a predicted AQI is not an official measurement or a health advisory. Users should rely on CPCB and other official sources and on qualified health guidance for decisions affecting health or safety.

17. Conclusion and Future Scope

17.1 Conclusion

This project developed AirAware, an educational prototype for next-day AQI forecasting. It prepared a daily Delhi AQI series from the India Air Quality dataset, built 30-day input sequences with a chronological split and leakage-aware scaling, and trained LSTM and GRU models alongside a persistence baseline. On the held-out test period the baseline had the lowest MAE (31.46) and the GRU the lowest RMSE (44.15), while the LSTM did not outperform the baseline. A Streamlit dashboard was built that validates uploaded data, shows trends and statistics, produces an experimental forecast with an indicative category, compares it with the baseline and provides a downloadable summary. The main lesson is that a simple baseline is a demanding benchmark for next-day AQI, and that honest comparison is as important as model sophistication.

17.2 Future Scope

The following are possibilities for future work and are not implemented features:

- extending the experiment to additional cities and comparing results across them;
- adding input features such as pollutant concentrations, weather variables and calendar effects;
- systematic hyperparameter tuning and repeated runs to measure variability;
- stronger baselines, such as seasonal or statistical time-series models, for comparison;
- uncertainty estimation, so forecasts are reported with intervals;
- validation on more recent external data and in other regions; and
- carefully validated integration of live data, with checks for missing and erroneous readings.

18. Tools and Technologies

Table 12 lists the tools used in the project, as evidenced by the notebook, the dashboard source and the dependency file.

Table 12: Tools and technologies used in AirAware.

Tool	Purpose in the project
Python	Programming language for the notebook and the dashboard
pandas [15]	Loading, cleaning and time-series handling of the AQI data
NumPy [16]	Array operations and sequence construction
scikit-learn [8]	MinMaxScaler and MAE/MSE evaluation functions
TensorFlow / Keras [13, 9]	LSTM and GRU model definition, training and saving (TensorFlow 2.21.0 in the notebook)
Matplotlib [17] and Seaborn [18]	Plots in the notebook
joblib	Saving and loading the fitted scaler
Streamlit [6]	Interactive dashboard
Jupyter Notebook / Google Colab	Development and execution environment for the experiment
Git and GitHub [7]	Version control and project repository
Cloudflare quick tunnel (cloudflared)	Temporary public access to the dashboard during the demonstration

The dependencies listed in `requirements.txt` are streamlit, pandas, numpy, scikit-learn, tensorflow and joblib; no version numbers are pinned in that file.

19. References

[1] Central Pollution Control Board, Government of India. *National Air Quality Index (AQI)*. https://cpcb.nic.in

[2] Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. https://otexts.com/fpp3/

[3] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780.

[4] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1724–1734).

[5] Rao, R. (n.d.). *Air Quality Data in India (2015–2020)* [Data set]. Kaggle. https://www.kaggle.com/datasets/rohanrao/air-quality-data-in-india

[6] Streamlit Inc. *Streamlit documentation*. https://docs.streamlit.io

[7] Maharana, P. P. *AirAware — Air Pollution Early Warning* [Source code repository]. GitHub. https://github.com/LEOPRITAM01/AirAware-Air_Pollution_Early_Warning

[8] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

[9] Chollet, F., et al. (2015). *Keras* and Keras API documentation. https://keras.io

[10] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*(56), 1929–1958.

[11] Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv:1412.3555*.

[12] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*. arXiv:1412.6980.

[13] Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)* (pp. 265–283).

[14] TensorFlow. *TensorFlow documentation*. https://www.tensorflow.org/

[15] McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61).

[16] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature, 585*, 357–362.

[17] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95.

[18] Waskom, M. L. (2021). seaborn: statistical data visualization. *Journal of Open Source Software, 6*(60), 3021.